



MASTER 1 INFORMATIQUE / GROUPE 3

OctoGenere

Conception d'une plateforme
d'évaluation de projets logiciels
assistée par un modèle de langage

Rapport de projet

SMUFN129 — Software Engineering & Design Patterns

Architecture modulaire, intégration des fournisseurs LLM,
exécution isolée et validation des résultats.

ÉQUIPE PROJET

Membre	Numéro étudiant
Alexandre BERTHO	22204437
Louis CIEBIERA	22205090
Gilles CONRAD	22607490
Reese DODE	22302249
Mathéo DARNAUDGUILHEM	22316680
Alice DURAND	22401928
Alexia GASTAUD	22307655
Sébastien LALLEMENT	22205593

Résumé

L'analyse d'un projet logiciel par un grand modèle de langage soulève des questions qui dépassent la simple formulation d'une requête : sélection du code pertinent, dépendance à un fournisseur, fiabilité des réponses, sécurité de l'exécution et restitution des résultats. OctoGenere répond à ces enjeux par une application Java organisée autour de contrats explicites entre ses composants.

Le prototype permet de sélectionner un répertoire local, d'en explorer l'arborescence, de choisir des critères et un modèle, puis d'obtenir une évaluation structurée. Une interface JavaFX pilote un moteur d'analyse indépendant de la présentation. Le code retenu est représenté dans un contexte XML borné ; les observations issues d'une sandbox Docker, lorsqu'elles sont disponibles, enrichissent le prompt. Les adaptateurs Gemini et OpenAI masquent les particularités des SDK. La réponse est validée avant toute restitution et peut être exportée dans un document LaTeX construit par l'application.

Le travail porte principalement sur la séparation des responsabilités et la justification des patrons de conception. Adapter, Strategy, Observer, Facade et Builder répondent à des problèmes identifiés ; une fabrique simple centralise la création des fournisseurs. Les critères sont décrits comme des données de configuration, ce qui évite d'introduire une classe par élément de notation.

La validation repose sur des tests de composants et des services simulés. La dernière exécution standard de Maven a recensé 109 tests, dont 104 réussis et 5 tests graphiques non exécutés. Ce résultat valide des comportements logiciels ; il ne mesure ni la pertinence pédagogique des notes attribuées par un LLM, ni la résistance complète de l'environnement d'exécution.

Contribution principale

Une chaîne d'analyse dont les frontières sont explicites : le moteur orchestre, les adaptateurs communiquent, la sandbox exécute, le validateur contrôle et l'interface restitue. Le modèle de langage apporte une appréciation, sans recevoir la responsabilité de piloter l'exécution locale.

Mots-clés : architecture logicielle ; MVC ; patrons de conception ; JavaFX ; modèles de langage ; Docker ; validation.

Périmètre documentaire

Ce rapport reprend le brouillon du groupe et décrit l'implémentation de la branche `dev` au commit `56e7ae8`, datée du 11 septembre 2026. Les intentions de conception sont distinguées des mécanismes effectivement présents. Les diagrammes représentent cette version du code ; les évolutions proposées sont annoncées comme telles.

Conventions de lecture

Terme	Signification dans le rapport
LLM	Grand modèle de langage utilisé pour apprécier le code.
Fournisseur	Service et SDK donnant accès à un ou plusieurs modèles.
Contexte	Ensemble des fichiers retenus et des informations jointes au prompt.
Sandbox	Environnement Docker réservé à l'exécution du projet analysé.
Rapport	Selon le contexte, le présent mémoire ou l'évaluation exportée par l'application.

Table des matières

Résumé	2
1 Problématique et périmètre fonctionnel	4
2 Méthodologie et choix techniques	5
3 Architecture logicielle	6
4 Patrons de conception et justification	8
5 Moteur d'analyse et intégration des LLM	10
6 Gestion des erreurs et restitution	12
7 Sécurité et exécution isolée	13
8 Stratégie de test et intégration continue	15
9 Extensibilité et perspectives	16
10 Bilan du projet	17
Annexe — Reproduction et repères techniques	18
Références	19

Organisation du document

La présentation suit les décisions de conception : besoin et organisation du travail, architecture et patrons, fonctionnement du moteur, sécurité, validation et perspectives. La transformation JSON–LaTeX est traitée de manière synthétique ; les enjeux d'architecture et d'intégration occupent l'essentiel du rapport.

1 Problématique et périmètre fonctionnel

1.1 Contexte et objectifs

Le sujet *AI Project Reviewer* demande de concevoir une plateforme d'évaluation de projets logiciels combinant analyses automatiques, recours à un LLM et production d'un rapport [1]. L'enjeu pédagogique est de construire une architecture extensible et explicable, tout en traitant le projet importé et les réponses du service externe comme des entrées non fiables.

Notre problématique est la suivante : *comment intégrer une évaluation probabiliste dans une application dont les comportements, les dépendances et les frontières de confiance restent maîtrisés ?* La réponse repose sur une séparation entre les opérations déterministes de préparation et de contrôle, et l'interprétation qualitative confiée au modèle. Un JSON conforme prouve que les résultats sont exploitables par l'application ; il ne prouve pas que leur contenu est exact.

1.2 Parcours utilisateur réalisé

L'utilisateur choisit un dossier local, consulte son arborescence et sélectionne les critères proposés. Il choisit ensuite un modèle du fournisseur configuré, lance l'analyse et suit sa progression dans l'interface. Après validation de la réponse, les scores et la synthèse sont affichés ; un bouton permet de produire un document LaTeX. Une modification du projet ou des paramètres par l'utilisateur invalide les résultats précédents afin d'éviter leur réutilisation involontaire.

Besoin	Réponse du prototype	État
Importer et explorer	Répertoire local et arbre de fichiers borné.	Réalisé
Configurer l'évaluation	Catalogue de critères et sélection du modèle.	Réalisé
Interroger un LLM	Adaptateurs Gemini et OpenAI; un fournisseur par configuration.	Réalisé
Suivre et contrôler	Tâches asynchrones, progression et erreurs visibles.	Réalisé
Exécuter le projet	Commande Maven dans un conteneur dédié, si l'environnement le permet.	Conditionnel
Restituer	Résultats validés et export LaTeX dans un dossier distinct.	Réalisé
Importer une archive ; conserver un historique	Pas d'import d'archive ni de service de consultation des analyses passées.	À développer

TABLE 1 – Couverture fonctionnelle de la version étudiée.

1.3 Exigences transversales

Trois propriétés orientent la conception : **testabilité**, afin de remplacer les services externes par des doubles contrôlés ; **réactivité**, afin que les appels réseau et les accès disque n'immobilisent pas JavaFX ; **maîtrise des entrées**, afin de limiter le volume de code transmis et de rejeter une réponse incohérente. L'extensibilité est appréciée au nombre de composants qu'il faut modifier pour introduire une capacité nouvelle, et non au nombre de patterns employés.

2 Méthodologie et choix techniques

2.1 Organisation du travail collectif

Le groupe a travaillé sur une période courte de deux jours. Le brouillon décrit une démarche itérative inspirée de l'agilité : identification des tâches sous forme d'issues, répartition du travail et échanges de conception. Il serait excessif de la présenter comme une application complète de Scrum ; les échanges ont surtout permis d'ajuster les contrats entre modules et de résoudre les difficultés d'intégration.

Le dépôt distingue `dev`, branche d'intégration, de `main`, destinée aux versions validées. Des branches de travail ont permis aux sous-groupes d'avancer sur l'IA, l'interface et la sécurité. L'intégration a notamment consisté à relier des composants d'abord développés séparément : construction du contexte, interrogation du LLM, validation et affichage. L'ajout du sélecteur de modèles illustre cette démarche : son comportement visuel a été complété par la découverte des modèles et la transmission de la sélection au fournisseur.

Pôle	Membres	Responsabilité collective
Intelligence artificielle	Mathéo Darnaudguilhem, Reese Dode, Alice Durand	Contexte du projet, prompt et échanges avec les modèles.
Interface JavaFX	Alexia Gastaud	Composants graphiques et parcours utilisateur.
Architecture et intégration	Alexandre Bertho, Louis Ciebiera, Gilles Conrad, Sébastien Lallement	Contrats, articulation des modules, sécurité, restitution et validation.

TABLE 2 – Répartition des pôles, d'après le README du groupe. Les travaux d'intégration sont transversaux.

2.2 Choix de réalisation

Technologie	Justification
Java 21 et Maven	Typage des contrats, objets de données immuables et construction commune à l'ensemble du groupe.
JavaFX 21	Arborescence, contrôles de sélection, tâches asynchrones et mise en forme CSS dans l'écosystème Java.
SDK Gemini / OpenAI	Encapsulation des protocoles API dans des adaptateurs dédiés.
Jackson	Lecture de la configuration et désérialisation contrôlée des résultats.
Docker	Exécution de commandes dans un environnement jetable, avec restrictions explicites.
LaTeX et JUnit 5	Production documentaire structurée et validation automatisée des composants.

2.3 Enseignement méthodologique

La difficulté principale a été de faire converger les hypothèses locales. Un menu peut être opérationnel visuellement sans modifier la requête réelle ; un prompt peut être lisible tout en contredisant le validateur. Les tests d'intégration et la relecture des contrats sont donc complémentaires des développements par sous-groupe. Les noms de classes et les schémas du présent rapport ont été rapprochés du code livré pour rendre ces décisions vérifiables.

3 Architecture logicielle

3.1 Une application modulaire à organisation MVC

OctoGenere est une application Java unique, organisée en packages. Le terme *module* désigne ici une séparation logique; il ne s'agit ni de microservices ni de modules Maven déployés séparément. L'interface suit une organisation MVC adaptée à JavaFX : les vues collectent les actions et présentent les données; `UIController` coordonne les traitements; les services métier réalisent l'exploration, l'analyse et l'export.

`MainApp` constitue le point d'assemblage. Il charge la configuration et le catalogue, crée le fournisseur et le moteur, puis relie le contrôleur aux panneaux. Cette construction explicite rend les dépendances visibles sans imposer un framework d'injection. Le moteur dépend de `LlmProvider` et de `AnalysisObserver`, pas d'un contrôle graphique concret.

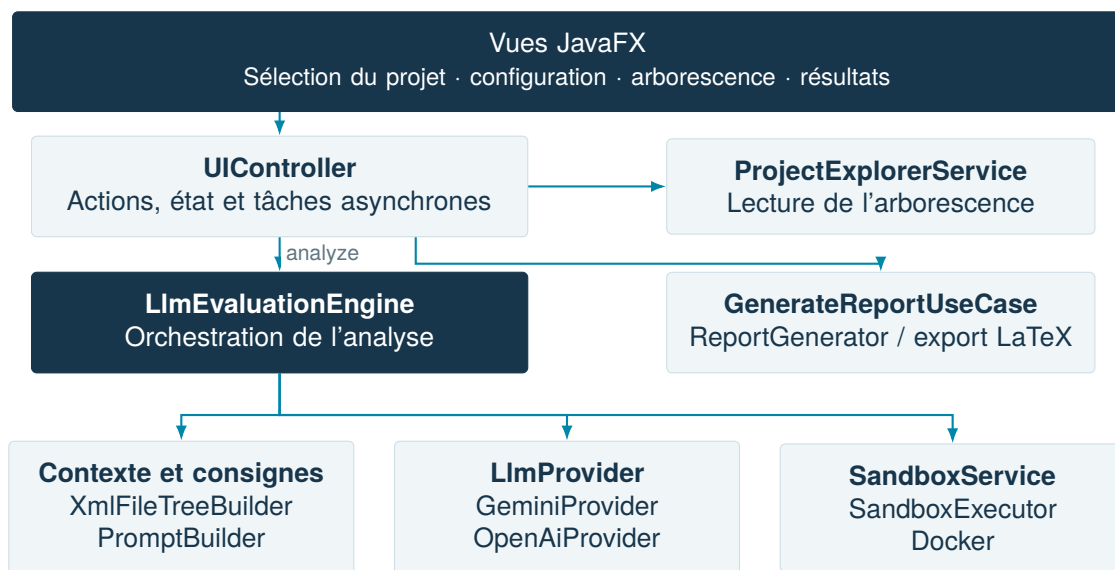


FIGURE 1 – Principales dépendances de service. Le contrôleur transmet à l'export le résultat métier renvoyé par le moteur; le fournisseur est le seul composant reliant l'analyse au service LLM distant.

3.2 Données et responsabilités

`EvaluationReport` regroupe l'identité du projet, la date, les paramètres descriptifs, le modèle, les résultats par critère et la synthèse. Chaque `CriterionResult` porte une note, son maximum et les observations associées. Ces records effectuent des contrôles de construction et des copies de collections, ce qui évite qu'une modification ultérieure d'une liste d'entrée altère un résultat déjà validé.

L'exploration visuelle relève de `ProjectExplorerService`, qui construit les nœuds de l'arbre. Le composant `XmlFileTreeBuilder` lit les contenus retenus pour le prompt. Ces deux parcours ont des finalités et des limites différentes; leurs représentations ne sont donc pas strictement identiques.

Repères dans le code : `ui/MainApp.java`, `ui/controller/UIController.java`, `packages/analysis/model`, `project` et `prompt/xml`.

Architecture logicielle — comportement dynamique

3.3 Une orchestration explicite de l'analyse

Le moteur prépare le contexte, tente l'exécution Maven dans la sandbox, construit le prompt puis appelle le fournisseur. Les flux standard et d'erreur de la sandbox deviennent des observations jointes à la requête. Le LLM reçoit du texte et produit une réponse : il ne choisit pas de commande à lancer et ne dialogue pas directement avec Docker.

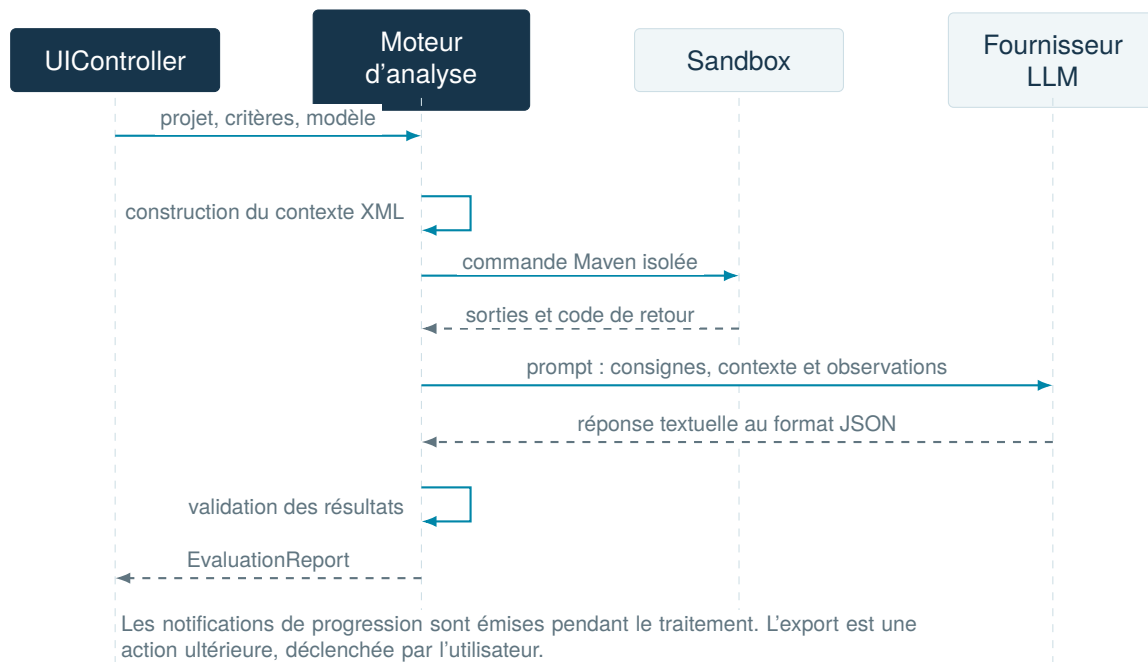


FIGURE 2 — Scénario nominal d'une analyse. En cas de *SandboxException*, le moteur poursuit avec une mention d'indisponibilité; une réponse LLM invalide interrompt la restitution des résultats.

3.4 Réactivité et cohérence de l'interface

Le contrôleur exécute l'exploration, l'analyse et l'export dans un exécuteur dédié. Des objets `Task` organisent les retours de succès, d'échec et d'annulation. Les notifications métier sont replacées sur le thread JavaFX par `Platform.runLater`, conformément au modèle de concurrence de JavaFX [4]. Un second exécuteur charge la liste des modèles indépendamment de ces opérations.

Pendant un traitement, les contrôles concernés sont désactivés. Le contrôleur ne conserve qu'un dernier rapport en mémoire et autorise son export après succès. Cette gestion empêche plusieurs actions concurrentes depuis les boutons, mais ne constitue pas un historique persistant. La synchronisation entre l'arrivée tardive de la liste des modèles et une analyse déjà lancée reste un cas à approfondir par des tests graphiques.

4 Patrons de conception et justification

Les patrons sont retenus pour résoudre un problème concret de variation ou de couplage. Nous utilisons la terminologie classique des patrons objets [3], en distinguant une implémentation complète d'une simple ressemblance structurelle.

4.1 Adapter : isoler les protocoles externes

Problème. Gemini et OpenAI n'exposent pas les mêmes classes de requête, structures de réponse et exceptions. Une dépendance directe du moteur envers ces API disperserait les particularités techniques dans la logique d'analyse.

Réalisation. GeminiProvider et OpenAiProvider réalisent LlmProvider. Ils construisent la requête propre au SDK, extraient le texte exploitable et traduisent les erreurs en LlmException. Le moteur peut donc appeler ask à travers un contrat commun. Le modèle choisi est un paramètre de la requête, distinct de l'identité du fournisseur.

Intérêt et limite. L'adaptation facilite le remplacement du SDK et les tests. Elle ne rend pas tous les modèles sémantiquement équivalents : formats supportés, quotas et capacités de génération restent dépendants du service.

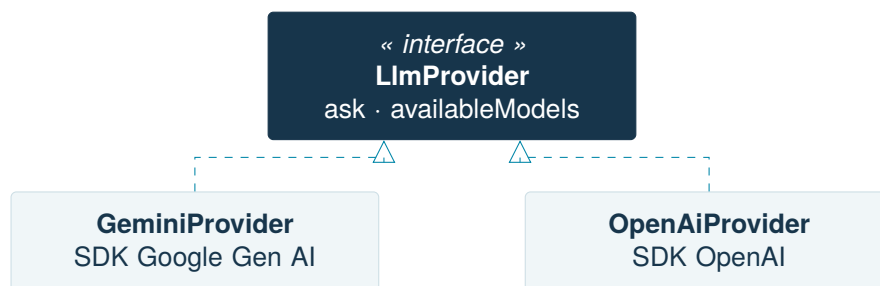


FIGURE 3 – Réalisation du contrat fournisseur par deux adaptateurs. Les flèches pointent vers l'interface implémentée.

4.2 Fabrique simple : centraliser la création

`LlmProviderFactory.from` lit `LLM_PROVIDER` et construit l'adaptateur correspondant. La sélection est concentrée dans un `switch`, au lieu d'être répétée dans l'interface et le moteur. La fabrique répond à la question *quel objet créer ?*, alors que l'adaptateur répond à *comment utiliser ce service à travers notre contrat ?* Les deux mécanismes sont complémentaires.

Il s'agit d'une **fabrique simple**, pas d'une Abstract Factory construisant des familles d'objets, ni d'un Factory Method fondé sur la redéfinition d'une méthode par héritage. L'ajout d'un fournisseur nécessite encore de modifier ce point de création; cette centralisation reste proportionnée à deux implémentations.

4.3 Strategy : substituer un comportement

`EvaluationEngine` permet au contrôleur de recevoir un moteur réel ou `DemoEvaluationEngine`. Ce dernier produit des résultats fictifs pour tester l'interface et l'export sans réseau. De même, `SandboxExecutor` permet à `SandboxService` de déléguer à Docker ou à un faux exécuteur. La substitution est réalisée par composition; elle ne suppose pas un sélecteur de stratégie exposé à l'utilisateur.

Patrons de conception — notifications et construction

4.4 Observer : notifier sans dépendre de JavaFX

Problème. Une analyse dure assez longtemps pour nécessiter un retour de progression, alors que le moteur doit rester utilisable hors de l'interface graphique.

Réalisation. `AnalysisObserver` expose `onProgressUpdate` et `onNewLogAdded`. Le contrôleur implémente ce contrat et fournit son instance à l'analyse. Le moteur émet les notifications sans connaître les panneaux. Le contrôleur assure ensuite leur restitution sur le thread de l'interface.

Compromis. Le mécanisme est une forme simple d'Observer, avec un observateur transmis par appel. Il ne comporte pas de registre d'abonnés ni de bus d'événements. Cette simplicité couvre le besoin actuel et facilite les tests : un observateur de test peut enregistrer les messages et vérifier que la progression atteint sa valeur finale.

4.5 Facade : proposer une entrée de service limitée

`SandboxService.run` offre une façade au sous-système d'exécution. L'appelant fournit un dossier et une commande ; il n'assemble pas les options de `docker run`, ne lit pas lui-même les flux du processus et ne gère pas son nettoyage. Ces détails sont pris en charge derrière `SandboxExecutor`.

La façade simplifie l'utilisation du sous-système ; `DockerSandboxExecutor` adapte ensuite l'outil en ligne de commande au contrat Java. Facade et Adapter interviennent donc à deux niveaux différents. Le choix des restrictions reste documenté et testable dans `DockerCommandBuilder` et `SandboxLimits`.

4.6 Builder : construire une demande d'exécution cohérente

`ExecutionRequest.Builder` associe deux éléments obligatoires, le répertoire et la commande, à des paramètres optionnels : limites, réseau et environnement. Il évite une multiplication de constructeurs difficiles à lire et produit une demande dont les listes et les associations sont copiées.

Ce Builder facilite l'évolution de la configuration sans obliger tous les appelants à renseigner chaque option. Il ne remplace pas une validation exhaustive de sécurité : les contraintes doivent toujours être appliquées par l'exécuteur. Les builders des SDK sont, quant à eux, des mécanismes fournis par ces bibliothèques, et non des patrons conçus par le groupe.

4.7 Des abstractions volontairement limitées

Les critères décrivent ici des libellés, des maxima et des paramètres d'activation. Une classe `Strategy` par critère aurait ajouté des objets sans encapsuler d'algorithme distinct. La solution retenue est un catalogue de données validé au démarrage, partagé avec le moteur et transmis à la vue.

L'arbre `FileNode` est une structure récursive : un dossier contient des enfants du même type. Il présente une parenté avec `Composite`, mais le code ne définit pas de hiérarchie explicite *composant/feuille/composé* avec opérations polymorphes. De même, un enchaînement fixe d'étapes dans `LlmEvaluationEngine` ne suffit pas à constituer `Template Method` : aucune étape n'y est spécialisée par héritage.

Bilan de conception

Le bénéfice des patterns est visible dans les tests et les points de substitution. La principale dette de couplage concerne la sandbox : le moteur crée encore directement sa façade par `createDefault()`, ce qui limite son remplacement au niveau du scénario d'analyse complet.

5 Moteur d'analyse et intégration des LLM

5.1 Des critères chargés une seule fois

`CriterionCatalog` lit la ressource `analysis/criteria.json`, contrôle ses propriétés et construit une liste de `EvaluationCriterion`. Les identifiants et libellés doivent être uniques ; le moteur résout ensuite les libellés transmis par la vue à partir de ce catalogue. Contrairement à une lecture directe depuis les cases à cocher, cette résolution conserve une référence métier unique pour les maxima et les critères attendus.

Critère configuré	Activation	Poids	Maximum
Qualité de l'architecture (SOLID)	Oui	0,25	10
Lisibilité et qualité du code	Oui	0,25	10
Gestion des exceptions	Oui	0,15	10
Qualité de la documentation	Non	0,15	10
Sécurité	Oui	0,20	10

TABLE 3 – Catalogue fourni avec le prototype. Le champ de pondération est déclaré, mais n'est pas utilisé par le moteur actuel pour calculer une note globale pondérée.

5.2 Sélection et représentation du code

`XmlFileTreeBuilder` parcourt le projet sans suivre les liens symboliques rencontrés et ignore plusieurs répertoires générés. Il retient les fichiers texte compatibles avec ses règles d'extension, exclut des noms et formats sensibles courants, puis décode les contenus en UTF-8. Les candidats retenus sont ordonnés par chemin relatif pour rendre leur présentation stable.

Borne	Objectif	Valeur
Fichiers inclus	Limiter le contexte transmis.	120
Taille d'un fichier	Écarter les fichiers trop volumineux.	64 Kio
Cumul des contenus source	Borner les données lues et retenues.	512 Kio
Candidats découverts	Arrêter une exploration excessive.	5 000

Les textes sont protégés par `PromptInjectionGuard`, puis placés dans une hiérarchie XML de dossiers et de fichiers. Ce XML est une représentation textuelle de transport : il est inséré dans le prompt envoyé au fournisseur, sans dépôt séparé ni serveur de fichiers intermédiaire. Les nombres de fichiers inclus et omis ainsi que l'indicateur de troncature sont remontés au moteur.

5.3 Compromis de gestion du contexte

Le prototype réalise une requête d'évaluation globale sur un sous-ensemble borné du projet. Il ne découpe pas le code en analyses multiples et n'agrège pas de synthèses intermédiaires. Ce choix limite les interactions réseau et la complexité de coordination, au prix d'une perte possible d'éléments pertinents.

La limite de 512 Kio concerne les contenus source avant ajout des balises, avertissements et journaux ; elle n'est pas un budget exact de tokens. De plus, lorsque la limite de découverte est atteinte, le tri ne porte que sur les candidats déjà rencontrés. Une sélection guidée par les dépendances du projet et un budget adapté au modèle constitueraient une amélioration substantielle.

Moteur d'analyse — requêtes et choix du modèle

5.4 Construction du prompt

PromptBuilder réunit un rôle de relecteur logiciel, le nom du projet, la liste des critères, une structure JSON attendue, les observations de la sandbox et le contexte XML. Les consignes demandent de juger le code contenu dans les fichiers, sans évaluer le XML comme s'il s'agissait de l'architecture réelle du projet. Les éléments de code sont annoncés comme des données non fiables.

Les journaux de compilation et de test sont placés après la fermeture de l'exemple JSON, afin de ne pas en altérer la syntaxe. Leur rôle est d'apporter des indices : une commande en échec ne prouve pas, à elle seule, un défaut du code. L'environnement, les dépendances indisponibles ou les ressources autorisées peuvent expliquer l'échec.

Un contrat de sortie à maintenir cohérent

Le contrat demande quatre champs à la racine : `score`, `maxScore`, `overallSummary` et `criteria`. Le prompt présente désormais cette même structure et les réponses simulées par les tests suivent ce format. Cet alignement évite qu'une réponse conforme aux instructions données au modèle soit ensuite rejetée par le validateur.

5.5 Fournisseur et modèle : deux niveaux distincts

Le fournisseur est choisi au démarrage par `LLM_PROVIDER`. Les adaptateurs encapsulent les SDK Google Gen AI et OpenAI. Le prototype utilise des API distantes ; il ne livre pas encore d'adaptateur dédié à un modèle local. Ce choix réduit la mise en place initiale, mais maintient une dépendance au réseau, à la clé API et aux conditions d'accès du service.

Pour Gemini, la requête demande une sortie de type `application/json`, avec une température de 0,2 et un plafond de 4 096 tokens de sortie. OpenAI utilise l'API Responses avec extraction de la réponse textuelle. Le paramétrage n'est donc pas identique entre les services ; la validation applicative reste commune. Une température faible peut réduire la variabilité, sans rendre les résultats déterministes.

5.6 Découverte des modèles dans l'interface

Le sélecteur affiche initialement le modèle de configuration. Pour Gemini, le provider interroge `models.list` via le SDK, parcourt les résultats et ne conserve que les entrées annonçant l'action `generateContent`, documentée par Google [5]. Les noms sont normalisés en retirant le préfixe `models/`, dédoublés et triés.

Si la liste distante est vide ou si son chargement échoue, `GOOGLE_MODEL` demeure le choix de repli. Si la liste est disponible, la sélection courante est conservée lorsqu'elle y figure ; sinon, le premier modèle proposé est choisi. Le chargement s'effectue dans un exécuteur distinct, sans immobiliser l'exploration du dossier.

Au lancement, la valeur choisie est copiée dans la tâche d'analyse, transmise au moteur puis à l'adaptateur. Elle est également inscrite dans le rapport métier. Pour OpenAI, la liste exposée reste limitée au modèle configuré. Enfin, la capacité `generateContent` ne garantit pas à elle seule la compatibilité d'un modèle avec tous les paramètres de notre requête, notamment le format JSON ; un échec demeure possible et doit être traité.

6 Gestion des erreurs et restitution

6.1 Contrôler les réponses à la frontière du système

Le moteur n'utilise la sortie du fournisseur qu'après validation. `EvaluationResponseParser` refuse notamment les champs inconnus, les propriétés obligatoires manquantes, les clés dupliquées, les données supplémentaires après le JSON et les conversions implicites de types. Il vérifie que chaque critère attendu apparaît une seule fois et que son maximum correspond au catalogue.

Les invariants des objets métier complètent ces contrôles : une note doit être entière et comprise entre zéro et le maximum ; les textes requis et les listes d'observations doivent respecter leur contrat. La réponse est réordonnée selon les critères demandés. Ainsi, l'interface et l'export utilisent le même résultat validé, au lieu de réinterpréter séparément une réponse brute.

Situation	Traitement	Conséquence visible
Configuration absente	Initialisation d'un moteur indisponible avec message explicatif.	Fenêtre accessible ; analyse refusée avec erreur.
Authentification, quota ou panne réseau	Traduction des exceptions du SDK en message métier.	Message d'erreur IA et possibilité de relancer.
Liste de modèles vide ou inaccessible	Conservation du modèle de configuration.	Avertissement et sélecteur utilisable.
Réponse vide, incomplète ou non conforme	Rejet par l'adaptateur ou le parseur.	Aucun nouveau résultat exportable.
<code>SandboxException</code>	Poursuite avec mention d'une analyse statique uniquement.	Avertissement dans les journaux.
Échec d'écriture du document	Propagation vers le contrôleur.	Erreur d'export, sans message de succès.

TABLE 4 – Politique de traitement des principaux échecs.

6.2 Résilience et traçabilité

Les contrôles sont réactivés après un échec afin de permettre une nouvelle tentative de l'utilisateur. La configuration applique un délai de 60 secondes pour Gemini et de 30 secondes pour OpenAI ; l'adaptateur OpenAI désactive les tentatives automatiques du SDK. Aucun mécanisme applicatif de reprise partielle, de temporisation exponentielle ou de bascule automatique vers un autre fournisseur n'est implémenté.

Les messages du moteur indiquent la préparation, les fichiers retenus, les omissions, le fournisseur, le modèle et les étapes de validation. Les erreurs SDK sont traduites sans afficher leur corps privé. Ces journaux sont destinés à la session courante ; ils ne constituent pas encore une trace persistante avec empreinte du contexte, identifiant de requête et durée de toutes les étapes.

6.3 Production du compte rendu LaTeX

`GenerateReportUseCase` reçoit le rapport métier validé et délègue à `ReportGenerator`. Son implémentation LaTeX applique un modèle documentaire et échappe les caractères réservés. Le LLM fournit des observations, tandis que l'application conserve la structure du document. Chaque export crée un sous-dossier de `target/reports`, contenant le fichier `rapport-evaluation.tex` et le logo. La compilation PDF de cette évaluation est une étape distincte ; elle n'est pas automatisée par le bouton d'export actuel.

7 Sécurité et exécution isolée

7.1 Deux frontières de confiance

Le sujet impose de considérer le projet importé comme potentiellement malveillant [1]. Deux risques doivent être séparés : l'exécution de code susceptible d'affecter la machine, et l'introduction de consignes malveillantes dans les données lues par le LLM. Docker réduit l'exposition liée à l'exécution ; la préparation du prompt et la validation réduisent certains risques liés à l'interprétation. Aucun de ces mécanismes ne constitue une protection universelle.

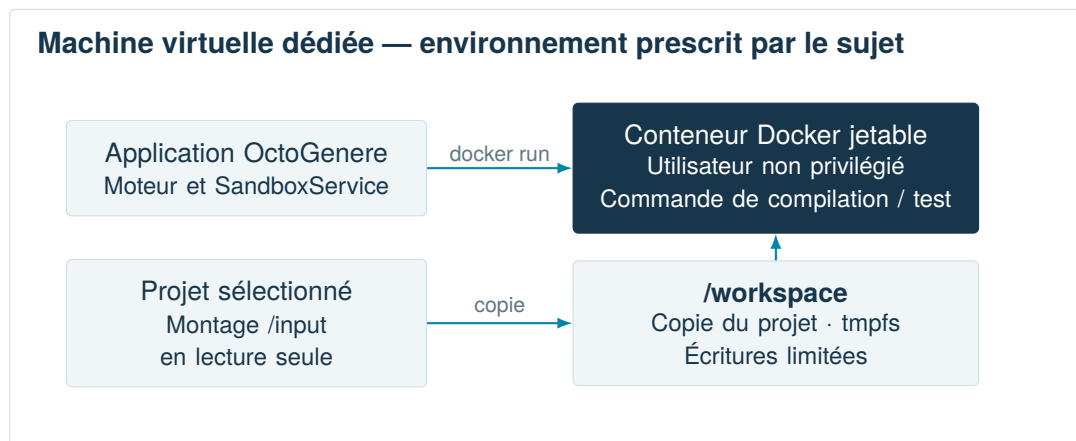


FIGURE 4 – Déploiement attendu pour l'exécution de projets non fiables. Les restrictions Docker sont codées ; la présence et le provisionnement de la machine virtuelle ne sont pas automatisés par le projet.

7.2 Restrictions effectivement construites

DockerCommandBuilder prépare les arguments de lancement sous forme de liste, sans concaténation d'une commande interprétée par un shell de l'hôte. Il applique un utilisateur non privilégié, un système de fichiers racine en lecture seule, la suppression des capacités et l'interdiction d'acquérir de nouveaux privilèges. Le projet est monté en lecture seule, puis copié vers un espace de travail temporaire.

Ressource / accès	Valeur par défaut	Intention
CPU ; mémoire	1 CPU ; 512 Mio	Contenir la consommation.
Processus ; durée	128 processus ; 120 secondes	Limiter prolifération et blocage.
Réseau	Désactivé	Réduire les communications sortantes.
Espace de travail	tmpfs de 256 Mio ; /tmp de 64 Mio	Limiter les écritures temporaires.
Compte du conteneur	UID / GID 10001	Éviter une exécution en root.

La documentation Docker souligne l'importance de définir explicitement des limites de ressources [6]. Leur présence dans les arguments peut être vérifiée sans exécuter de projet inconnu ; leur efficacité réelle doit aussi être éprouvée dans l'environnement de déploiement.

Sécurité — garanties et limites

7.3 Cycle de vie de la sandbox

Chaque exécution reçoit un nom de conteneur distinct. L'exécuteur lit séparément les flux standard et d'erreur et ajoute un délai de surveillance Java au délai interne du conteneur. Un nettoyage est tenté dans un bloc `finally`, en complément de l'option de suppression automatique de Docker. Cette tentative peut échouer : elle ne doit pas être présentée comme une garantie absolue d'absence de processus résiduel.

Le retour inclut le code de sortie, les sorties textuelles, une durée et l'indication de dépassement de délai. Le code 137 est traité comme un arrêt forcé ; il ne distingue pas à lui seul une limite mémoire d'un dépassement de durée. La version du moteur utilise principalement les sorties et le code de retour pour construire le prompt.

7.4 Réduire les injections de prompt

`PromptInjectionGuard` combine un scanner heuristique de consignes suspectes et un mécanisme de nettoyage et de délimitation. Ce dernier retire certains caractères invisibles, encadre les fichiers par des marqueurs explicites et neutralise les occurrences de ces marqueurs dans le contenu. Le texte suspect est conservé comme objet d'analyse et signalé, plutôt que silencieusement supprimé.

Cette stratégie donne au modèle des indications sur l'origine et le statut du contenu. Elle reste heuristique : une instruction peut être obfusquée ou reformulée, et le respect des consignes par un LLM n'est pas garanti. La validation de sortie reste indispensable, tout en étant incapable de reconnaître à elle seule une appréciation biaisée mais formellement valide.

7.5 Confidentialité et surfaces restantes

Le filtre de fichiers exclut notamment `.env`, plusieurs noms de fichiers d'identifiants et des extensions de clés. Il s'agit d'une protection par catégories, et non d'un détecteur exhaustif de secrets : des identifiants inscrits dans un fichier source ou dans `.env.example`, autorisé comme exemple de configuration, peuvent encore être transmis. Le contenu retenu quitte l'application lors d'un appel à une API distante.

Les journaux de sandbox constituent une autre entrée non fiable. Dans cette version, ils sont ajoutés au prompt sans réutiliser la protection appliquée aux fichiers source. Leur collecte n'impose pas non plus de plafond explicite sur les flux lus dans la mémoire Java. Un projet très bavard peut donc accroître le volume du prompt ou la consommation de mémoire hors du conteneur. Une limite de collecte, un filtrage des secrets et un encadrement équivalent à celui des fichiers sont nécessaires pour durcir cette frontière.

7.6 Conditions d'exécution et limites expérimentales

Le réseau désactivé empêche Maven de télécharger des dépendances absentes de l'image. Un échec de compilation peut donc résulter du confinement. Le prototype ne provisionne pas automatiquement un cache de dépendances validé, ni une machine virtuelle dédiée. Cette dernière reste une condition de déploiement à mettre en place pour respecter le scénario prescrit par le sujet.

En cas de `SandboxException`, l'application poursuit la lecture statique sans exécuter la commande sur l'hôte. D'autres défaillances Docker peuvent être retournées comme un code non nul : il reste à distinguer explicitement indisponibilité d'infrastructure et échec des tests du projet. Enfin, aucune campagne d'attaque dans une machine virtuelle n'est attestée par les tests standard présentés dans ce rapport.

8 Stratégie de test et intégration continue

8.1 Des tests indépendants du service distant

Les tests OpenAI utilisent le SDK réel face à un serveur HTTP local simulé; ceux de Gemini injectent des fonctions de génération et de découverte contrôlées. Le test du moteur fournit une réponse LLM déterminée pour vérifier le contexte, la sélection du modèle et l'export. Ces tests ne nécessitent aucune clé API réelle.

Les tests de sécurité contrôlent les arguments Docker et l'exécution avec des doubles, ce qui les distingue des essais d'un vrai conteneur. Le moteur créant sa sandbox directement, son test peut tenter Docker, puis emprunter le repli statique si l'outil est absent. L'injection de cette dépendance rendrait ce scénario entièrement contrôlé.

Famille	Réussis	Comportements vérifiés
Configuration et objets métier	4	Catalogue, invariants et validité des données.
Moteurs et parseurs	22	Réponses attendues ou invalides, scénario d'analyse.
Fournisseurs LLM	24	Texte, erreurs HTTP, délai, modèle transmis, découverte.
Projet et contexte XML	6	Parcours, sélection des fichiers et limites.
Génération du rapport	4	Export, modèle documentaire et échappement.
Protection du prompt	19	Détection heuristique et délimitation du contenu.
Composants de sandbox	25	Configuration, restrictions, exécution simulée.
Total réussi	104	Aucun échec ni erreur dans la suite standard.

TABLE 5 – Résultats de la dernière validation standard sur la version étudiée : 109 tests recensés, dont 5 tests JavaFX non exécutés. Les tests Docker nommés *IT ne sont pas inclus dans ce total.

8.2 Portée de la validation

Les cinq tests graphiques requièrent une session graphique et une activation explicite. Les essais Docker se lancent séparément. Les compteurs du tableau proviennent des rapports Surefire; ils ne constituent ni un taux de couverture du code ni une preuve de sécurité en conditions hostiles.

Une évaluation qualitative des appréciations du LLM sur des projets de référence reste nécessaire. Le choix des modèles, l'arrivée tardive de leur catalogue et les sorties de sandbox volumineuses méritent aussi une campagne dédiée.

8.3 Intégration continue sur GitHub

Le workflow `.github/workflows/test.yml` prévoit une validation sur les push et pull requests visant `main` ou `dev`, ainsi qu'un lancement manuel. Il prépare Java 21 et exécute `mvn verify`. Le cache Maven accélère les exécutions; les droits sont limités à la lecture du dépôt. La configuration impose un délai maximal de quinze minutes et annule un ancien travail concurrent pour le même contexte de branche. Cette description porte sur le workflow versionné, sans préjuger du résultat d'une exécution distante particulière.

9 Extensibilité et perspectives

9.1 Points d'extension du prototype

L'extensibilité est inégale selon les fonctionnalités. Les contrats couvrent déjà le fournisseur, le moteur et l'export; la sélection des fichiers et la commande d'exécution restent plus directement codées dans leurs composants. Le tableau suivant précise le coût d'une extension sans supposer l'existence d'un mécanisme de plugins.

Évolution	Point d'intervention	Limite actuelle
Nouveau fournisseur	Implémenter <code>LlmProvider</code> , adapter requêtes et erreurs, puis compléter la fabrique.	La création reste centralisée dans un <code>switch</code> .
Nouveau critère	Ajouter une entrée à <code>criteria.json</code> et ses paramètres.	Ressource embarquée, rechargée au démarrage après mise à jour du build.
Nouveau format de rapport	Implémenter <code>ReportGenerator</code> et modifier l'assemblage ainsi que les libellés de l'UI.	Aucun choix de format dans la fenêtre actuelle.
Nouvelle règle de sélection	Modifier <code>XmlFileTreeBuilder</code> , ou extraire un contrat de politique de fichiers.	Règles codées en dur; parcours visuel distinct.
Nouvelle méthode d'analyse	Implémenter <code>EvaluationEngine</code> avec le même résultat et le même observateur.	Pas de composition générale de plusieurs analyseurs.
Nouveau langage exécuté	Adapter l'image et la commande de sandbox au système de construction.	Le moteur lance actuellement une commande Maven.

TABLE 6 – Évolutions possibles et modifications nécessaires.

9.2 Priorités d'amélioration

Stabiliser les contrats. La première priorité est d'aligner le prompt avec le schéma effectivement validé et de vérifier la compatibilité des modèles proposés avec le format de sortie demandé. La cohérence entre configuration affichée, paramètres de la tâche et rapport doit être testée lorsque la liste distante arrive en cours d'utilisation.

Durcir les frontières d'entrée. La collecte des journaux doit être bornée, filtrée et protégée contre les injections. Une préparation contrôlée des dépendances Maven et une distinction claire entre panne d'infrastructure et échec des tests rendraient les observations plus interprétables. L'environnement VM/Docker doit être préparé et testé explicitement.

Étendre la couverture fonctionnelle. L'import d'archives, un stockage persistant des analyses et l'accès à un LLM local répondraient à des besoins du sujet encore non couverts. La gestion de grands projets appelle ensuite un découpage du contexte et une consolidation des résultats intermédiaires. Ces mécanismes doivent préserver les preuves utilisées pour chaque appréciation.

Mesurer la qualité des évaluations. Une pondération effective, un jeu de projets étalons et des comparaisons répétées entre modèles permettraient d'étudier la stabilité des notes. La conformité JSON ne peut tenir lieu de validation scientifique des jugements produits.

10 Bilan du projet

10.1 Apports de la réalisation

Le projet a transformé une intention d'*analyse de code par IA* en responsabilités identifiables. L'interface, le moteur, le service externe et la production documentaire sont reliés par des contrats. Cette séparation facilite le développement parallèle et les tests sans dépendre d'une réponse réelle de modèle.

L'évolution des schémas a clarifié plusieurs ambiguïtés : le contrôleur graphique coordonne les actions, le moteur orchestre l'analyse, la sandbox exécute et l'export reçoit des données validées. Cette clarification permet de situer une erreur et d'identifier le composant chargé de la traiter.

Le sélecteur de modèles illustre ce découpage. La vue collecte une préférence, le contrôleur la copie dans une tâche, le moteur l'associe à l'analyse et l'adaptateur la transmet au SDK. Le rapport conserve le modèle demandé, sans que la vue connaisse la structure d'une requête Gemini.

10.2 Compromis et enseignements

La contrainte de temps a conduit à privilégier un parcours complet sur un répertoire local plutôt qu'une couverture exhaustive du sujet. Le prototype traite un contexte borné, conserve un résultat en mémoire et exporte du LaTeX. Il ne propose pas encore d'historique consultable, de fournisseur local livré, d'analyse distribuée ni de compilation PDF intégrée à l'interface.

Les design patterns ont surtout été utiles lorsqu'ils correspondaient à une variation réelle : adapter un SDK, substituer un moteur, recevoir une notification ou masquer la mécanique de Docker. À l'inverse, la configuration des critères montre qu'une modélisation par données peut être plus appropriée qu'une nouvelle hiérarchie de classes. La testabilité fournit ici un critère concret pour apprécier la pertinence d'une abstraction.

Une fenêtre réactive, une réponse valide, un export réussi et une appréciation pertinente sont quatre propriétés différentes. La dernière exige un protocole d'évaluation humain et expérimental qui dépasse les tests unitaires actuels.

10.3 Usage des outils d'assistance

Conformément au sujet, des outils d'assistance par IA ont été mobilisés pour accompagner l'intégration, l'analyse d'erreurs, la vérification d'API et la préparation documentaire. Ils ont également contribué à la rédaction et à la mise en forme du présent rapport à partir du brouillon du groupe et du code. La responsabilité des explications, des choix techniques et de la validation finale demeure collective. Aucune contribution individuelle supplémentaire n'est déduite du seul usage de ces outils.

Conclusion

OctoGenere fournit une base fonctionnelle pour l'évaluation assistée de projets logiciels. Son apport essentiel est une architecture explicite autour d'un composant probabiliste, avec des points de substitution et de validation. La poursuite du travail doit désormais porter sur la cohérence des contrats, la maîtrise des sorties non fiables et la mesure de la qualité des évaluations.

Annexe — Reproduction et repères techniques

Lancer le prototype

Les commandes suivantes sont exécutées à la racine du dépôt, avec un JDK 21 et Maven. Copier le modèle de configuration dans `.env`, puis y renseigner la clé et un identifiant de modèle valide pour le fournisseur choisi. Le fichier d'exemple partagé ne doit contenir aucune clé réelle.

```
cp .env.example .env
mvn javafx:run
```

Dans la fenêtre, sélectionner le projet et les critères, vérifier le modèle retenu, puis lancer l'analyse. Une fois les résultats disponibles, le bouton d'export crée un dossier distinct sous `target/reports`. La documentation d'installation complémentaire figure dans le README du dépôt [2].

Valider les composants

```
mvn --batch-mode --no-transfer-progress verify
```

Le résultat standard documenté est de 109 tests recensés : 104 réussis et 5 non exécutés. Les rapports XML sont produits dans `target/surefire-reports`. Le fichier `docs/rapport/validation-tests.json` en conserve une synthèse numérique pour la version décrite, sans les propriétés d'environnement des rapports bruts.

Les tests graphiques doivent être activés dans une session graphique :

```
mvn -Doctogenere.ui.tests=true -Dtest=UiWorkflowTest test
```

Préparer les essais Docker

Pour l'exécution de projets non fiables, préparer au préalable la machine virtuelle dédiée prescrite par le sujet, puis y construire l'image et lancer les essais d'intégration :

```
docker build -t octogenere/sandbox:1.0 -f docker/Dockerfile docker
mvn -Dtest=fr.octogenere.security.sandbox.DockerSandboxExecutorIT test
```

Ces commandes sont un protocole de reproduction ; leur présence dans cette annexe ne signifie pas que l'environnement VM/Docker a été validé lors de la préparation du rapport.

Recompiler le présent rapport

Avec XeLaTeX, latexmk et les polices TeX Gyre installés :

```
latexmk -xelatex -interaction=nonstopmode -halt-on-error \
  -outdir=target/rapport Rapport-OctoGenere.tex
cp target/rapport/Rapport-OctoGenere.pdf Rapport-OctoGenere.pdf
```

Le document est distinct d'une évaluation automatiquement générée par l'application. Le logo institutionnel est inclus dans les ressources du rapport et les schémas sont construits en vectoriel avec TikZ ; aucune récupération réseau n'est nécessaire à la recompilation.

Références

- [1] Université Côte d’Azur, Master Informatique. *Project : AI Project Reviewer — Design of an Automated Evaluation Platform for AI-Assisted Software Projects*. Sujet de projet, année universitaire 2026–2027. Document fourni dans le dépôt : docs/sujet2627-m1-en.pdf. En particulier, sections 6, 8, 10, 11 et 13.
- [2] Groupe 3. *OctoGenere : code source, documentation et tests*. Branche dev, version 56e7ae8, 11 septembre 2026. <https://github.com/alexbertho/octogenere/tree/56e7ae8>. Les descriptions d’implémentation et les résultats de test du rapport se rapportent à cette version.
- [3] Erich Gamma, Richard Helm, Ralph Johnson et John Vlissides. *Design Patterns : Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994. ISBN 978-0-201-63361-0. Référence terminologique pour les patrons de conception objets.
- [4] OpenJFX. *JavaFX 21 API — Task*. Documentation officielle du modèle de tâches et des contraintes du thread JavaFX. <https://openjfx.io/javadoc/21/javafx.graphics/javafx/concurrent/Task.html>. Consultée le 11 septembre 2026.
- [5] Google. *Gemini API — Models*. Documentation officielle des opérations `models.list` et `models.get` et des fonctionnalités annoncées par les modèles. <https://ai.google.dev/api/models>. Consultée le 11 septembre 2026.
- [6] Docker. *Resource constraints*. Documentation officielle des limites de ressources des conteneurs. https://docs.docker.com/engine/containers/resource_constraints/. Consultée le 11 septembre 2026.
- [7] Université Côte d’Azur, EUR DS4H. *Communication | Médias — Identité graphique de l’EUR DS4H*. <https://ds4h.univ-cotedazur.fr/a-propos-de-leur-ds4h/communication-medias-1>. Logo institutionnel téléchargé le 11 septembre 2026.

Crédits iconographiques

Le bandeau de couverture provient de la page de communication officielle de l’EUR DS4H [7]. Il est reproduit dans ses proportions et ses couleurs d’origine, avec les éléments institutionnels associés. Les schémas du corps du rapport ont été redessinés à partir des composants et interactions du code, en remplacement des représentations préliminaires du brouillon.